

Lambda

Introduzione

Lambda è una piccola funzione utile quando vogliamo definire una funzione in maniera veloce e semplice, ed è spesso usata come argomento di altre funzioni.

Sintassi

La sintassi di lambda è la seguente:

```
lambda arguments: expression
```

Quindi keyword lambda l'argomento e dopo i due punti l'espressione che andiamo a definire.

Come lavora lambda

Una lambda può avere un qualsiasi numero di argomenti, ma ha una sola espressione che ritorna in modo automatico.

Tipizzare lambda

Come facciamo a tipizzare gli argomenti e il valore di ritorno di lambda?

Per tipizzare lambda in [Python](#) si deve importare dal modulo `typing` il tipo `Callable` questo perché:

`Callable` non è built-in (come `int`, `str`, `list`, etc.) ma viene dal modulo `typing`, introdotto per supportare l'hinting statico dei tipi.

Inoltre le `lambda` sono funzioni anonime, e quindi sono oggetti di tipo `function`, cioè oggetti `Callable`.

In sostanza un oggetto `Callable` è un qualsiasi oggetto che può essere chiamato con le parentesi tonde `()` come una funzione:

```
from typing import Callable
def foo():
    print("ciao")
foo() # <-- foo è callable
```

Una `lambda` si comporta allo stesso modo:

```
from typing import Callable
f = lambda x: x * 2
f(3) # 6
```

Quindi per tipizzare `lambda` si scrive:

```
from typing import Callable
Callable [[ArgType, ArgType2, ...], ReturnType]
```

Spiegazione della sintassi:

1. `Callable` :
 - È il tipo che rappresenta una funzione `Callable` (richiamabile).
2. **lista nestata** `[[...]]` :
 - Contiene i tipi dei parametri in ordine
 - Esempio: `[[int, str]]` significa che la funzione accetta un intero e una stringa.
3. **Prima lista** `[]` :
 - il `ReturnType` specifica **il tipo del valore restituito**.
 - Esempio: se ci fosse il tipo `str` significherebbe che la funzione ritorna una stringa.

☰ Metafora

Possiamo vedere a `Callable[[A,B], C]` come:
Una funzione che prende `A` e `B` come input e restituisce `C`.

Esempio Pratico

```
from typing import Callable
multiply:Callable [[int,int], int] = lambda a,b: a*b
print(multiply(5,2))
```

Spiegazione:

In questo esempio:

1. `multiply` è una **variabile tipizzata come funzione** (`Callable`) che:
 - prende **due argomenti di tipo** `int`

- restituisce un valore di tipo `int`
2. Alla variabile `multiply` assegniamo una **funzione anonima** (lambda) che calcola il **prodotto dei due input**: `a * b`

L'utilità di `lambda`

In `Python`, `lambda` è una funzione anonima (si intende una funzione che non ha nome a meno che non venga assegnata ad una variabile) che si può scrivere in una sola riga. È utile soprattutto quando si vuole definire una funzione "al volo", da usare una o poche volte, senza doverle dare un nome esplicito.

Quindi `lambda`, rispetto alle normali funzioni di Python, ha una scrittura rapida, una sintassi diretta (per le callback semplici), una maggiore compattezza nelle operazioni inline e un'immediatezza per le chiusure semplici delle operazioni.

Tuttavia bisogna usarla con giudizio:

se da una parte le lambda sono strumenti per casi d'uso specifici, e sono utili per scrivere funzioni in modo compatto e semplice, dall'altra possono compromettere la leggibilità e la manutenibilità del codice, che deve essere sempre al primo posto e guidare le scelte prese sul codice.

Per usare le `lambda` ci sono alcune **best practices** da seguire:

1. Regola del pollice:

Formulazione della Regola:

```
"""Usa `lambda` solo se la funzione può essere scritta
comodamente in _un'unica espressione_ e non ha bisogno di un
nome.
Per tutto il resto, usa una funzione tradizionale (`def`)."""
```

Quindi si usa `lambda` quando:

- ✅ L'operazione è **così semplice** che non merita una funzione dedicata:
 - Ovvero la si usa una **sola volta**
 - La logica contenuta in un'unica espressione (senza if complessi, multi-linea, etc.)
- ❌ **Evitare** `lambda` **quando**:
 - La logica richiede **più espressioni** (`if / else` multipli, `loop`)
 - Si necessita di **documentazione**(docstring) o **type hints** completi
 - La funzione viene **riutilizzata** in più parti del codice

👉 Regola del pollice (Rule of Thumb)

In italiano la regola del pollice può essere tradotta anche come *regola empirica* o *regola pratica*, è un'espressione che indica una regola generale o linea guida basata sull'esperienza piuttosto che su calcoli scientifici o teorie formali.

✓ In breve:

Una "rule of thumb" è un criterio semplice, spesso non perfetto, ma utile per prendere decisioni rapide o pratiche in una situazione comune.

🔗 Regole del pollice comuni in programmazione >



1. Funzioni brevi e con un solo scopo

Se la funzione supera le **20-30 righe**, probabilmente fa troppo

Perché:

- Mantieni ogni funzione focalizzata su una **sola responsabilità** (principio SRP-Single Responsibility Principle).
- Più facile da testare leggere e riutilizzare



2. Evita la duplicazione

Se **copi e incolli** il codice **più di due volte**, estrailo in una funzione.

Perché:

- Il codice duplicato è difficile da leggere
- Segui il principio di **DRY = Don't Repeat Yourself**
- Meno ripetizioni equivalgono a meno bug



3. Commenta perché, non cosa

Il codice dovrebbe spiegarsi da solo.

I commenti si usano solo per spiegare il **perché** si fa qualcosa, non cosa si sta facendo

Perché:

- Nomi chiari per funzioni e variabili > Commenti inutili.
- Esempio:

```
# Evitiamo divisione per zero
if denominator != 0:
    result = numerator / denominato
```



4. Non ottimizzare prematuramente

Prima si fa funzionare il codice → poi lo si pulisce → solo dopo lo si ottimizza

Perché:

- Ottimizzazioni premature = tempo perso e codice complicato
- Concentrati prima sulla **chiarezza e correttezza**



5. Massimo 3 livelli di indentazione

Se il codice ha più di 3 livelli di `if`, `for`, etc. va semplificato

Perché:

Troppa indentazione = difficile da leggere



Bonus: Regole del pollice specifiche per Python

- ☒ Usare `lambda` solo per funzioni **semplici e usa-e-getta**
- ☒ Usare 'list comprehension' per trasformazioni rapide su liste

- ⚠ Non abusare degli `import *`: rende il codice poco chiaro
- ⚠ Non esagerare con le one-liner: **leggibilità > brevità**

2. Type hinting

3. Alternativa moderna:

Le espressioni di assegnazione possono talvolta sostituire le lambda:

```
# Invece di:  
filtered = filter(lambda x: x > 0, values)  
  
# Puoi scrivere:  
filtered = (x for x in values if x > 0)
```

4. Performance

Esempio pratico di `lambda`:

```
from typing import Callable  
square: Callable[[int], int] = lambda x: x**2  
print(square(5)) #Output 25  
#La controparte come funzione normale  
def square (x: int):  
    return x**2
```

Analisi:

1. Importazione del tipo `Callable`

```
from typing import Callable
```

2. Lambda Function con Type Hints:

```
square: Callable[[int], int] = lambda x: x**2
```

- Qui stiamo definendo una funzione lambda che:
 - Prende un parametro `x` di tipo `int` (specificato in `Callable[[int], ...]`)

- Restituisce un `int` (specificato in `Callable[... , int]`)
- Il corpo della lambda è `x**2` (calcola il quadrato di `x`)
- L'assegnazione a `square` permette di richiamarla come una normale funzione.

3. Chiamata della lambda

```
print(square(5)) # Output: 25
```

- Quando chiamiamo `square(5)` , la lambda calcola `5**2` e restituisce `25` .

4. Versione con Funzione Tradizionale:

```
def square(x: int) -> int:  
    return x**2
```

- Questa è l'equivalente definizione con `def` :
 - `x: int` indica che il parametro `x` è di tipo `int`
 - `-> int` specifica che il ritorno è un `int`
 - Il corpo della funzione fa esattamente la stessa cosa: calcola `x**2`

Differenze Chiave: >

1. Sintassi:

- La lambda è definita in una singola espressione.
- La funzione tradizionale può contenere più espressioni e statement.

2. Type Hints:

- Per la lambda, gli hint sono esterni (`Callable[[int], int]`).
- Per la funzione, gli hint sono incorporati nella definizione .

3. Nome:

- La lambda è tecnicamente anonima (ma le assegniamo a `square`).
- La funzione ha un nome (`square`) che appare negli stack trace.

Quando Usare Ciascuna Versione:

- **Lambda:** Ideale per operazioni semplici e usa-e-getta, specialmente come argomenti per altre funzioni (es. `map` , `filter`).

- **Funzione tradizionale:** Preferibile quando:
 - La logica è più complessa
 - Serve riutilizzare la funzione
 - Vogliamo documentazione (docstring)
 - Abbiamo bisogno di type hints più leggibili

? Quando usare `lambda`

✓ Casi d'uso

- Quando la funzione è semplice e usata solo una volta (es: in `map`, `'filter'`, `'sorted'`, etc.)
- Quando si vuole mantenere il codice più compatto e leggibile

⚡ Quando non usare `lambda`

- Se la funzione è complessa o la si riutilizza in più punti del codice
- Nel caso in cui si voglia aggiungere documentazione (docstring) o debug più facilmente

Esempio 2: moltiplicazione di due variabili

Usiamo `lambda` per moltiplicare due valori di due variabili:

```
from typing import Callable
multiply:Callable[[float,float],float] = lambda a,b: a*b
print(multiply(3,4)) #Output 12
```

Analisi del codice:

1. Importazione del tipo Callable:

```
from typing import Callable
```

Come prima, si importa `Callable` dal modulo `typing`, per poter **specificare il tipo** della nostra funzione.

2. Lambda Function con Type Hints:


```
multiply: Callable[[float, float], float] = lambda a, b: a * b
```

Qui stiamo:

- Definendo una **funzione** `lambda` **anonima** che prende due argomenti `a` e `b`, e restituisce il loro prodotto (`a * b`).
- Assegnando questa funzione alla variabile `multiply`.
- Indicando esplicitamente il **tipo** della funzione:
 - Prende **due** `float` come argomenti → `Callable[[float, float], ...]`
 - Restituisce un `float` → `..., float]`

3. Chiamata della lambda

```
print(multiply(3, 4)) # Output: 12
```

Qui stiamo **chiamando la funzione** `multiply` passando `3` e `4` come argomenti.

Python calcola: `3 * 4 = 12` e stampa `12`.

Anche se `3` e `4` sono interi (`int`), vengono automaticamente **convertiti in** `float` **se necessario**, quindi questo tipo di dichiarazione funziona anche con numeri interi.

☰ In sintesi:

- ☒ `lambda a, b: a * b` → definisce una funzione "al volo" che moltiplica due numeri.
- ☒ Il `Callable[[float, float], float]` specifica che la funzione:
 - Prende **due numeri decimali** (o interi compatibili)
 - Restituisce un numero decimale (`float`)

Utilizzo di `if` statements nella `lambda`

Le funzioni `lambda` possono includere **espressioni condizionali** (ternary operator), ma non supportano statement condizionali completi (`if-elif-else` multilinea).

La sintassi è sempre della forma:

```
lambda args: valore_se_vero if condizione else valore_se_falso
```

Componenti analizzate:

1. Keyword `lambda`

- Segnala a Python che stiamo definendo una funzione anonima.

2. Argomenti (`args`)

- Possono essere uno o più parametri (separati da virgole se multipli).

3. Corpo della Lambda

- `valore_se_vero` : Il valore restituito se `condizione` è `True` .
- `if condizione` :
 - Deve essere un'espressione che restituisce `True / False` (es: `x > 0, is_valid`).
- `else valore_se_falso` :
 - Obbligatorio in Python (a differenza di altri linguaggi).
 - Restituito se `condizione` è `False` .

Esempio Concreto

```
from typing import Callable
positive_or_negative:Callable[[int],int] = lambda x : "Positivo" if
x >0 else "Zero o Negativo"
print(positive_or_negative(5))
print(positive_or_negative(-3))
#Controparte
def positive_or_negative(x:int) ->str:
    if x > 0:
        return "Positivo"
    else:
        return "Negativo"
```

Spiegazione passo-passo:

1. Importazione del tipo `Callable`:

```
from typing import Callable
```

- Importiamo il tipo `Callable` dal modulo `typing` per aggiungere annotazioni di tipo alla nostra funzione lambda.

2. Definizione della lambda con type hints:

```
multiply: Callable[[float, float], float] = lambda a, b: a * b
```

- `Callable[[float, float], float]` specifica che:
 - La funzione accetta due parametri di tipo `float`
 - Restituisce un valore di tipo `float`
- `lambda a, b: a * b` è la funzione anonima che:
 - Prende due parametri `a` e `b`
 - Restituisce il loro prodotto `a * b`

3. Assegnazione a una variabile:

Assegniamo la lambda alla variabile `multiply`, che ora può essere chiamata come una normale funzione.

4. Chiamata e output:

```
print(multiply(3, 4)) # Output 12
```

- Chiamiamo `multiply` con argomenti 3 e 4
- La lambda calcola $3 * 4 = 12$
- `print` mostra il risultato

Lambda come argomento delle funzioni

Finora abbiamo visto come definire funzioni `lambda` da usare nel codice. Ma le `lambda` sono particolarmente utili **quando vengono passate come argomenti ad altre funzioni**.

Un esempio classico è la funzione `filter()`:

Filtrare una lista con la funzione `Filter()`

```
num: list[int] = [1, 2, 3, 4, 5, 6]
even: list[int] = list(filter(lambda x: x % 2 == 0, num))
print(even)
```

Tramite la funzione `filter` (che torna uno oggetto di tipo `filter`), filtriamo la lista `num` per ritornare una nuova lista (`even`) di numeri interi pari.
In altre parole questa funzione built-in - Accetta due argomenti:

1. Una funzione di filtro (in questo caso una lambda)
2. Un iterabile (la lista `numeri`).

Quindi:

- Abbiamo una lista di numeri `num`.
- Usiamo `filter()` per **selezionare solo i numeri pari**.
- La funzione `lambda x: x % 2 == 0` viene passata come **argomento** a `filter()`.
 - Per ogni elemento `x` della lista `num`, `lambda` restituisce `True` se `x` è pari.
- `filter()` applica questa logica a tutti gli elementi e **mantiene solo quelli per cui la condizione è vera**.
- Convertiamo il risultato in una lista con `list()`.

In questo caso **non è necessario importare** `Callable`, perché non stiamo **assegnando** la `lambda` a una variabile, ma la stiamo **usando direttamente** come parametro.

Il vantaggio di usare `lambda` con `Filter()` è che su una sola riga si possono filtrare una lista

Ordinamento Personalizzato con `sorted()` e Lambda

La funzione built-in `sorted()` in Python permette di ordinare sequenze in modo personalizzato attraverso il parametro `key`.

Le funzioni lambda sono particolarmente utili in questo contesto per definire criteri di ordinamento complessi in modo conciso.

```
names:list[str] = ["Alice", "bob", "Charlie"]
sorted_by_lenght:list[str] =sorted(names, key=lambda name:len(name))
print(sorted_by_lenght)
```

In questo caso si usa la funzione `sorted()`, che accetta:

- come **primo argomento**, la lista da ordinare (`names`)
- il parametro `key`, che indica **in base a cosa ordinare gli elementi**
- La funzione `lambda name: len(name)` dice a `sorted()`:

“Per ogni elemento `name` nella lista, usa la sua lunghezza (`len(name)`) come criterio per l'ordinamento.”

In particolare il parametro `key` :

- Accetta una funzione che viene applicata ad ogni elemento prima dell'ordinamento
- L'ordinamento avviene in base ai valori restituiti da questa funzione
- Nel nostro caso: `len(name)` calcola la lunghezza di ogni nome

`lambda name: len(name)` è equivalente a:

```
def get_length(name: str) -> int:  
    return len(name)
```

Se la si volesse riordinare al contrario:

```
names: list[str] = ["Alice", "bob", "Charlie"]  
sorted_by_lenght: list[str] = sorted(names, key=lambda name: len(name),  
reverse=True)  
print(sorted_by_lenght)
```

Bisogna inserire il parametro `reverse= True` , inverte l'ordine

Lambda e RegEx: Combinazione potente per elaborazione testi

L'integrazione tra funzioni lambda e espressioni regolari ([RegEx](#)) in Python permette di scrivere codice conciso ed espressivo per l'elaborazione di stringhe.

Esempio 1: Filtrare Elementi con `filter()` e RegEx

```
import re  
words: list[str] = ["abc123", "456", "43", "hello", "98abc",  
"test999"]  
  
# Filtra solo le stringhe composte interamente da cifre  
only_digits = list(filter(lambda x: re.fullmatch(r"\d+", x), words))  
print(only_digits) # Output: ['456', '43']
```

Meccanismo:

- `re.fullmatch(r"\d+", x)` verifica che l'intera stringa (`fullmatch`) contenga solo cifre (`\d+`)

- `filter()` applica la lambda a ogni elemento, mantenendo solo quelli dove la condizione è `True`
- La lambda funge da **funzione di validazione** inline

Fullmatch: controlla che il match rispetti esattamente il pattern

lambda definisce la condizione inline

`filter()`: applica lambda a tutte le stringhe nella lista.

L'equivalente Dinamica con funzione tradizionale

```
def is_all_digits(s: str) -> bool:
    return bool(re.fullmatch(r"\d+", s))
```

Esempio 2: Sostituzione Dinamica con `re.sub()` e Lambda

```
import re
text:str = "Price: 100 dollars, Tax:20 dollars"
new_text:str = re.sub(r"\d+", lambda m: str(int(m.group())*2), text)
print(new_text)
```

Componenti Chiave:

1. `re.sub()` cerca tutte le occorrenze del pattern `\d+` (numeri)
2. Per ogni match:
 - `m.group()` restituisce la stringa matchata (es. `"100"`)
 - La lambda converte a intero, raddoppia il valore, e riconverte a stringa
3. Il risultato viene sostituito nel testo originale

Dettagli Importanti:

- `m` è un oggetto `re.Match` che rappresenta il risultato della corrispondenza
- Le lambda in `re.sub()` permettono **elaborazioni complesse** delle sostituzioni

Lambda e la funzione `Reduce()`

La funzione `reduce()` è una funzione del modulo `functools` in Python, difatti va importata esplicitamente:

```
from functools import reduce
```

Definizione:

Questa funzione applica accumulativamente (o anche iterativamente) una funzione a due argomenti agli elementi di un iterabile, da sinistra a destra, riducendoli a un singolo valore.

Sintassi della funzione `reduce()`

La sintassi di questa funzione è:

```
from functools import reduce

reduce(funzione, iterabile[, valore_iniziale])
```

Dove:

- `funzione` : una funzione che prende due argomenti e restituisce un valore
- `iterabile` : una lista (o un altro oggetto iterabile) su cui lavorare.
- `valore_iniziale` (**opzionale**): un valore da usare come punto di partenza.

I casi d'uso comuni di questa funzione sono:

- Somme
- Prodotti
- Concatenazioni
- Composizioni di funzioni

Esempi concreti: Somma di una lista

```
from functools import reduce

numeri = [1, 2, 3, 4]
somma = reduce(lambda a, b: a + b, numeri)
print(somma)

# Passaggi:
# 1. 1 + 2 = 3
# 2. 3 + 3 = 6
# 3. 6 + 4 = 10 → Risultato finale
```

Spiegazione:

- Abbiamo una lista di numeri → `numeri`

- assegniamo la funzione `reduce()` alla variabile `somma`, come argomento di `reduce()` mettiamo:
 - Come `funzione`: una `lambda` che prende esattamente 2 soli argomenti (`a` e `b`) e come espressione di `lambda` si fa eseguire il calcolo tra `a` e `b`.
 - Come `iterabile`: la lista `numeri`; in sostanza viene detto alla funzione `reduce()` di iterare su ogni elemento presente nella lista `numeri`.

Fatto questo vediamo come funziona `reduce()`:

1. Prima iterazione:

- Prende i primi due elementi della lista (`1` e `2`)
- Applica la funzione `lambda`: `1 + 2 = 3`
- *Stato interno*: risultato parziale = `3`

2. Seconda iterazione:

- Prende il risultato precedente (`3`) e il terzo elemento (`3`)
- Applica la funzione: `3 + 3 = 6`
- *Stato interno*: risultato parziale = `6`

3. Terza iterazione:

- Prende il risultato precedente (`6`) e il quarto elemento (`4`)
- Applica la funzione: `6 + 4 = 10`
- *Risultato finale*: `10`

Visualizzazione grafica

```
Iterazione 1: [1, 2, 3, 4] → lambda(1, 2) → 3
Iterazione 2: [3, 3, 4]   → lambda(3, 3) → 6
Iterazione 3: [6, 4]     → lambda(6, 4) → 10
```

Requisiti Chiave

1. La `funzione`:

- Accetta **esattamente 2 parametri**
- Restituisce un valore compatibile con il tipo degli elementi.
- In questo caso `a` è il risultato accumulato, `b` è il prossimo elemento

2. L'iterabile deve contenere almeno 1 elemento (nel caso in cui non si usi un inizializzatore).